

多智能体规模、拓扑与合并负担

从局部 Error Rate 到图结构性能函数的实验路线

Agent Expressional Graph

2026 年 4 月 21 日

一句话总结

本次形成的核心问题是：在 Count Frequency 等可拆分任务中，增加 agent 数量会降低单个 agent 面对的局部问题规模，从而可能降低局部 error rate；但同时也会增加跨 agent 合并、对齐和聚合的负担。接下来的实验目标，是寻找 agent 数量、通信 topology 与 merge burden 之间的平衡点，并尝试建立从图结构到最终 error 的函数关系。

1 背景与动机

当前讨论围绕一个直观但关键的 trade-off 展开。对于一个固定规模的任务，如果使用更多 agents，每个 agent 分到的子问题会变小。

例如在 Count Frequency 问题中，数组被切成更多 subarray 后，每个 agent 只需要处理更少的数字。这样理论上会降低单个 agent 内部出错的概率，因为局部上下文更短、局部计数或分类判断更简单。

但这个收益并不是免费的。agent 数量增加后，系统末端需要合并更多局部结果，merge 的复杂度、通信量和错误传播路径都会增加。

因此，问题不只是“agent 越多越好吗”，而是：

局部问题规模下降带来的收益 是否能够超过 全局合并负担上升带来的损失。

2 核心问题

核心研究问题

是否存在一个理论平衡的 agent 数量 N^* ，使得局部求解误差和最终 merge burden 之间达到较优折中？进一步地，这个平衡点是否依赖于通信 topology？

可以把最终系统误差拆成两个来源：

$$e_{\text{final}} = e_{\text{local}} + e_{\text{merge}} + e_{\text{propagation}},$$

其中：

- e_{local} ：agent 在自己局部数组或局部子任务上的错误；

- e_{merge} : 多个局部结果在合并时产生的遗漏、重复计数、冲突或格式不一致;
- $e_{\text{propagation}}$: 错误信息在 topology 上传播后被其他 agents 接受或放大的部分。

增加 agent 数量通常会降低 e_{local} , 但可能提高 e_{merge} 和 $e_{\text{propagation}}$ 。因此, 一个可操作的实验目标是寻找:

$$N^* = \arg \min_N [e_{\text{local}}(N) + e_{\text{merge}}(N, \tau) + e_{\text{propagation}}(N, \tau)],$$

其中 τ 表示通信 topology。

3 DIG 视角: 哪些 Agents 真正有用

提出了一个自然切入点: 给定一组 agents 运行任务后, 不只看最终答案, 还要观察系统形成的交互图结构。关键问题是:

在一个 MAS run 中, 真正对最终答案有贡献的 agents 有几个? 这些 agents 在图中处于什么位置?

这实际上接近 DIG (Dynamic Interaction Graph) 的思想: 把每轮交互、消息传递、合并和最终贡献都记录为图上的结构信号。这样可以从图中抽取特征, 例如:

- 有效参与的 agent 数量;
- 最终答案依赖的局部结果数量;
- merge 节点的 fan-in;
- 信息传播路径长度;
- 是否存在少数高贡献 agents;
- 错误是否从某些局部节点扩散到全局结果。

图结构问题

如果把 topology 或运行后形成的交互图记为 E , 把最终 error 记为 e , 我们希望收集大量 pair:

$$(E, e),$$

并学习或拟合一个从图结构到 error 的函数:

$$e = f(E).$$

进一步地, 也可以把问题反过来: 如果已经观测到某种 error pattern, 例如局部 error 很低但最终 error 很高, 或者少数 agents 的错误被全局放大, 那么能否近似构造一个逆向映射:

$$E \approx f^{-1}(e),$$

4 从图到 Error 的建模路径

本次讨论中，最困难但也最有研究价值的问题是如何建立从图结构到 error 的函数。直接学习 $e = f(E)$ 比较难。GNN 可以但是需要尝试拓展 GNN 之外的方法。

5 下一阶段实验 Setup

下一步重点观察 Count Frequency (CF) 问题。初始设置如下：

- 全局数组规模： $M = 5000$ ；
- 数字范围：1 到 1000；
- agents 只看到分配给自己的 subarray；
- 尝试不同 topology；
- 每轮进行一次 merge 或 aggregation；
- 每轮记录总体 error rate 和每个 agent 的局部 error rate；
- setup 参考 MANCET 论文中的多智能体评测范式：固定任务族，控制 agent 数量、轮数和 topology，逐轮记录系统状态。

表 1: 实验控制变量

变量	取值	目的
Agent 数量 N	2, 4, 8, 16, ...	观察规模与合并负担的 trade-off
通信轮数 R	2, 3, 5	观察短程通信是否足够降低误差
数组长度 M	5000	固定任务总规模
数字范围	1 到 1000	固定类别/值域规模
Topology τ	chain, star, mesh, exponential 等	比较不同通信结构
Merge 频率	每轮一次	观察逐轮合并带来的收益和损失